FlashSAC

Fast and Stable Off-Policy Reinforcement Learning for High-Dimensional Robot Control

Outstanding Paper Award · RSS 2026

Donghu Kim*, **Youngdo Lee***, Minho Park, Kinam Kim,
Takuma Seno, I Made Aswin Nahrendra, Sehee Min, Daniel Palenicek,
Florian Vogt, Danica Kragic, Jan Peters, Jaegul Choo, Hojoon Lee

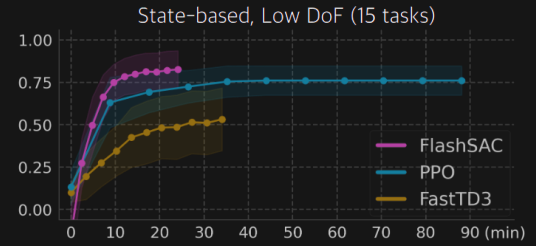
Holiday Robotics · KAIST · KRAFTON · Turing Inc · TU Darmstadt · hessian.AI
KTH Royal Institute of Technology · German Research Center for AI (DFKI)

Robert Teal

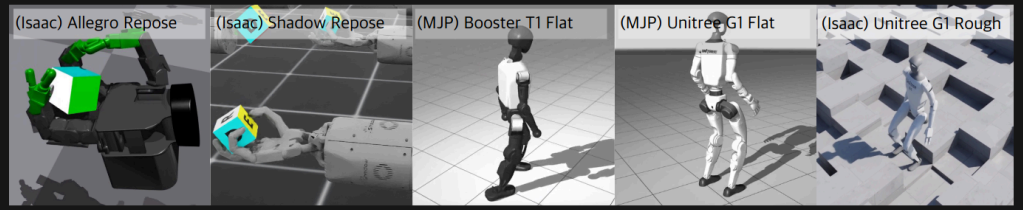
Fall 2026 Reading Group

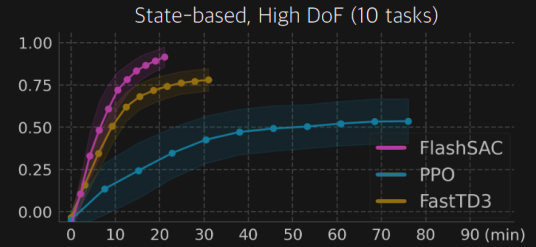
Results Preview

(a) State-based, Low DoF (Gripper Manipulation, Quadruped Locomotion)

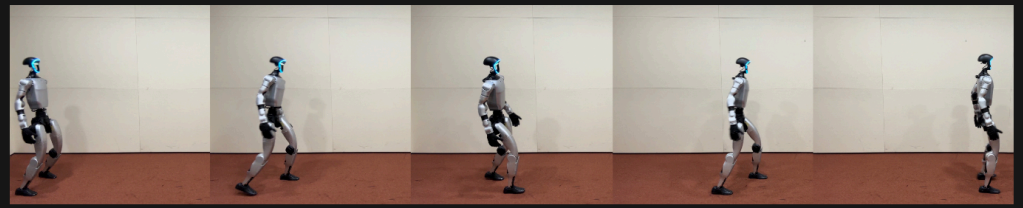


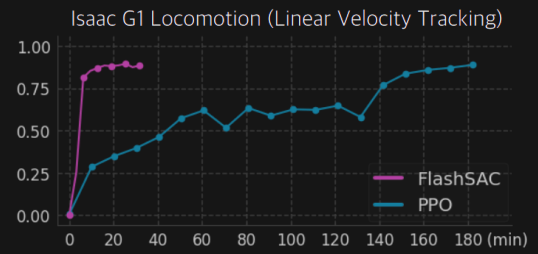
(b) State-based, High DoF (Dexterous Manipulation, Humanoid Locomotion)





(c) Sim-to-Real Reinforcement Learning (Humanoid Locomotion)





On-Policy, Until the Task Gets Large

Still a good fit

Quadrupeds and gripper arms. Low dimension, cheap parallel simulation, and a wasted batch barely matters.

No longer the typical task

Humanoids, dexterous hands, and vision. The on-policy batch is narrow, and simulation is more expensive.

> Throwing that batch away is then the bottleneck, in samples and in wall-clock time.

Off-Policy, Why It Stalls

The coverage is the point

Replay mixes many past policies, so the critic sees far more than the latest rollout.

The critic is the weak link

Fitting that diversity takes many updates, and each target is built from the critic itself.

$$(s, a, r, s') \in \mathcal{D}, \qquad a' \sim \pi(\cdot \mid s')$$

$$\mathcal{L}_Q = \mathbb{E}_{\mathcal{D}}\Big[\big(Q_\theta(s,a) - \big[r + \gamma Q_\theta(s', a')\big]\big)^2\Big]$$

On-Policy, Recent Work

PPO is the modern standard later on-policy methods are built on:

- Collect experience (s, a, r, s') with a fixed rollout policy π_{old}
- Estimate advantages $\hat{A}_t$ and perform several minibatch epochs using L^{clip}
- Replace the rollout with fresh experience

$$L^{\text{clip}} = \mathbb{E}_t\left[\min\left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right], \quad \rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

Off-Policy, Recent Work

- Recent work on off-policy methods fixes one bottleneck at a time:

Speed

FastTD3 and FastSAC push throughput, with networks around 0.2M parameters.

Stability

CrossQ, SimBaV2, and XQC bound the critic, then need many updates.

Exploration

Pink noise, OU noise, and action repeats make motion coherent.

- **FlashSAC** is SAC with all three: fewer updates, a bounded critic, and cheap correlated noise

Markov Decision Process

- Robotic control is modeled as a discounted MDP

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma) \qquad s \in \mathcal{S},\ a \in \mathcal{A},\ P(s' \mid s, a),\ r(s, a),\ \gamma \in [0, 1)$$

- Each step observes s_t, samples a_t, receives r_t, and draws s_{t+1}
- $\pi(a \mid s)$ maximizes the discounted return $J(\pi)$

$$\max_{\pi} J(\pi) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right]$$

Soft Actor-Critic (SAC)

- Store (s, a, r, s') from past policies in replay buffer $\mathcal{D}$, and train from it
- Entropy regularization, weighted by $\alpha > 0$, encourages exploration
- **Clipped double Q-learning:** Two critics $Q_{\phi 1}$ and $Q_{\phi 2}$

$$\mathcal{L}_\pi(\theta) = \mathbb{E}\left[\alpha \log \pi_\theta(a \mid s) - \min_{i=1,2} Q_{\phi_i}(s, a)\right]$$

Soft Actor-Critic (SAC)

- Target critics $\bar{\phi}_j$ track the online critics with a slow exponential moving average

$$\bar{\phi}_j \leftarrow \tau\phi_j + (1-\tau)\bar{\phi}_j, \qquad j \in \{1,2\},\ \tau \in (0,1)$$

- Critic weights are optimized by minimizing Bellman loss, with target y

$$\mathcal{L}_{Q_i} = \mathbb{E}_{\mathcal{D}}[(Q_{\phi_i}(s,a) - y)^2]$$

$$y = r + \gamma \left[\min_{j=1,2} Q_{\bar{\phi}_j}(s', a') - \alpha \log \pi_\theta(a' \mid s') \right], \quad a' \sim \pi_\theta(\cdot \mid s')$$

1. Fast Training

> Under a fixed compute budget, larger models and larger batches with fewer updates converge faster than small models updated often

Massive Parallel Simulation

1,024 environments

Large Replay Buffer

10M transitions vs. 1M

Larger Model

2.5M parameters vs. 0.2-0.5M
6 layers vs. 2–3

Large Batch

2,048 batch size

Fewer Updates

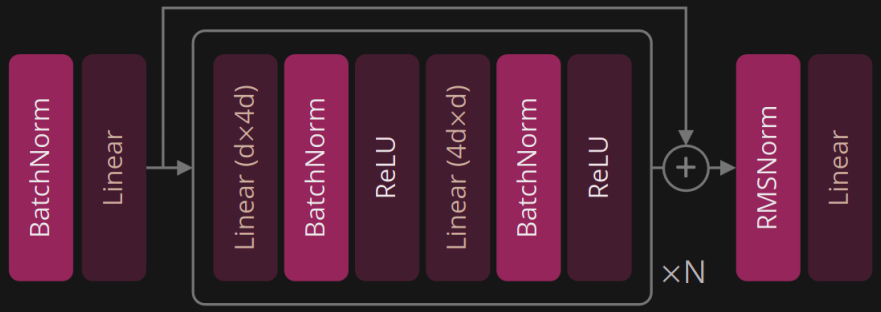
2 updates / 1,024 transitions

Code Optimization

JIT compiled and mixed precision

2. Stable Training

> Bound weight, feature, and gradient norms so bootstrap errors do not compound.



Inverted residual backbone

Expand, project back, residual. RMSNorm after the last block.

Pre-activation batch norm

Normalize before each nonlinearity.

2. Stable Training

> Bound weight, feature, and gradient norms so bootstrap errors do not compound.

- **Cross-batch targets:** current $Q(s, a)$ and next state $Q(s', a')$ share one batch
- **Distributional critic:** categorical Q on $[G_{\min}, G_{\max}]$, with normalized rewards

$$\bar{r}_t = \frac{r_t}{\max\left(\sqrt{\sigma_{t,G}^2 + \epsilon},\ G_{t,\max}/G_{\max}\right)}$$

- **Weight normalization:** project w to unit length, and normalization parameter vectors (γ, β) to norm $\sqrt{d}$

3. Exploration

> One noise scale for every robot, and action noise that persists across steps.

Unified entropy target: Parameterize the target entropy via $\sigma_{\text{tgt}} = 0.15$

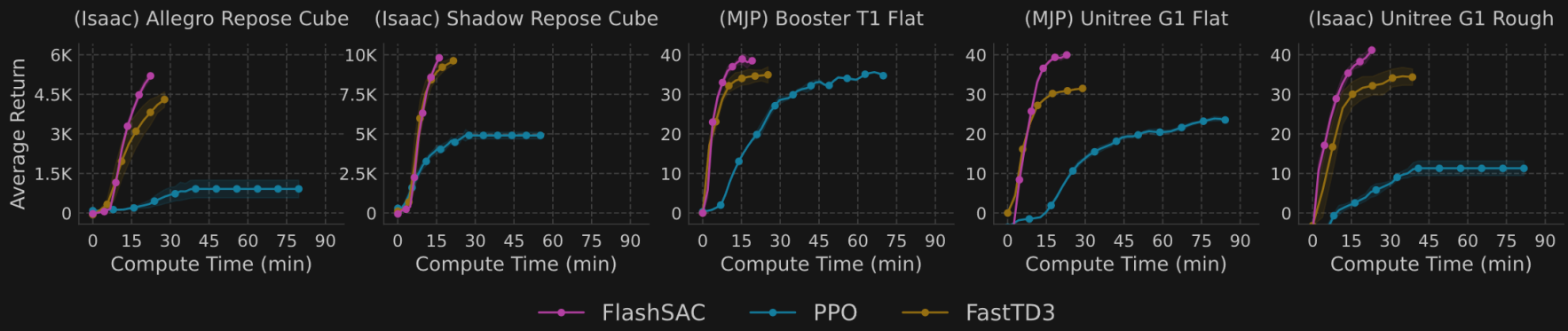
$$\bar{\mathcal{H}} = \frac{|\mathcal{A}|}{2} \log\left(2\pi e\, \sigma_{\text{tgt}}^2\right)$$

Noise repetition: Sample $\epsilon \sim \mathcal{N}(0, I)$ and hold it for k steps. Draw k from a zeta distribution, capped at 16:

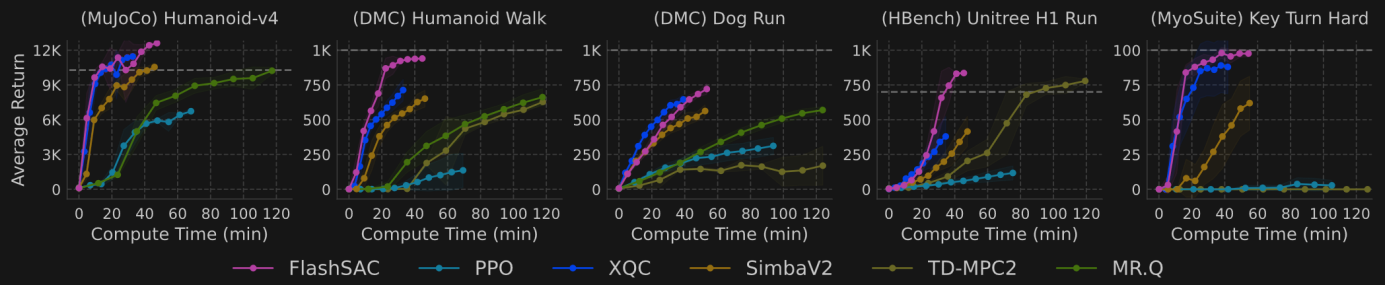
$$P(k) \propto k^{-s}, \qquad s = 2, \qquad k \in \{1, \ldots, 16\}$$

GPU-Based Simulators (Low Dimensional)

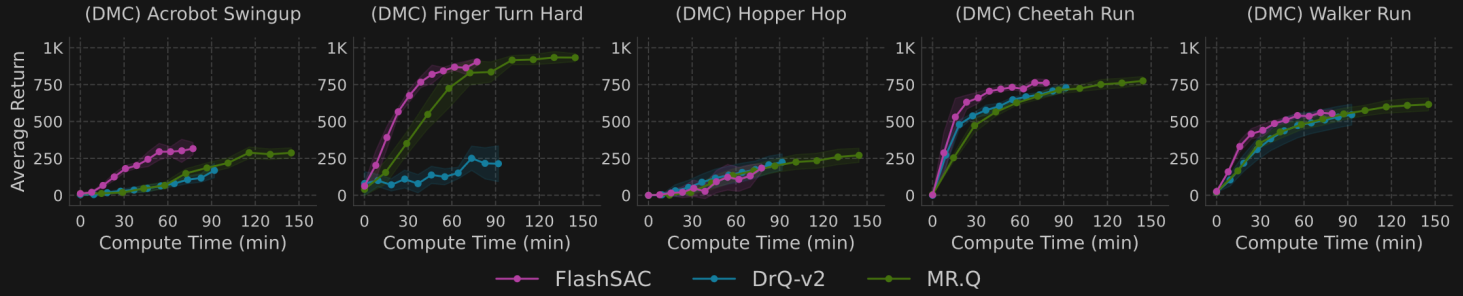
GPU-Based Simulators (High Dimensional)



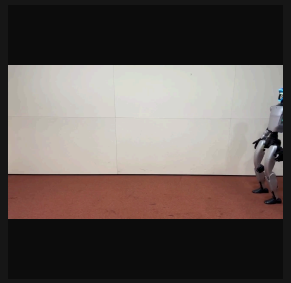
CPU-Based Simulators



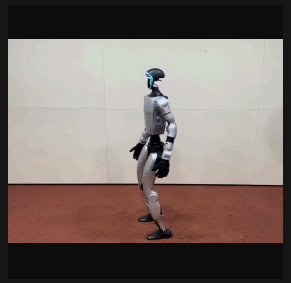
Vision-Based Simulators



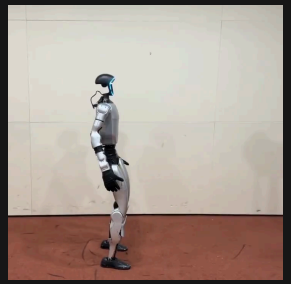
Sim-to-Real Transfer



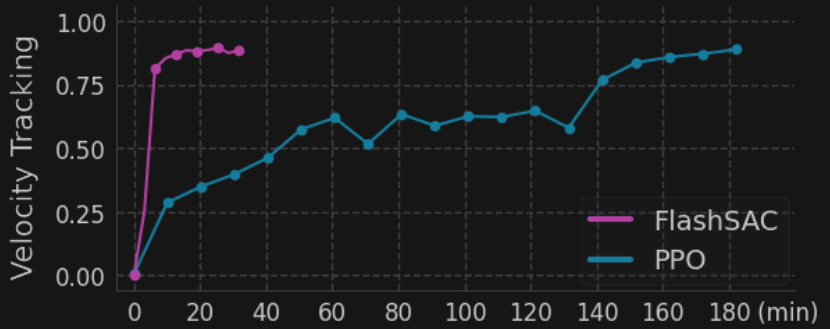
G1 Walking

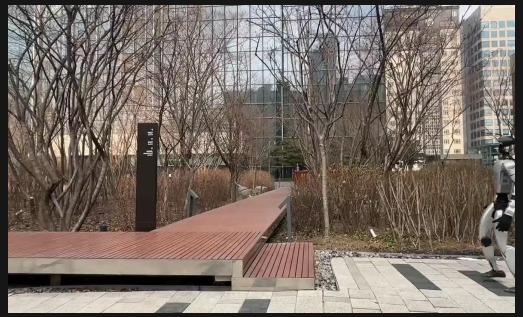


G1 Turning

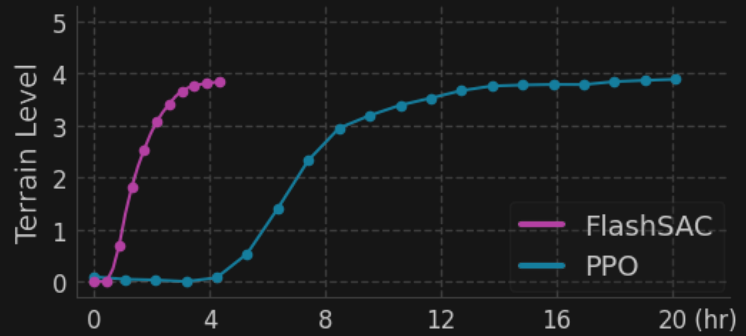


G1 Push Perturbation



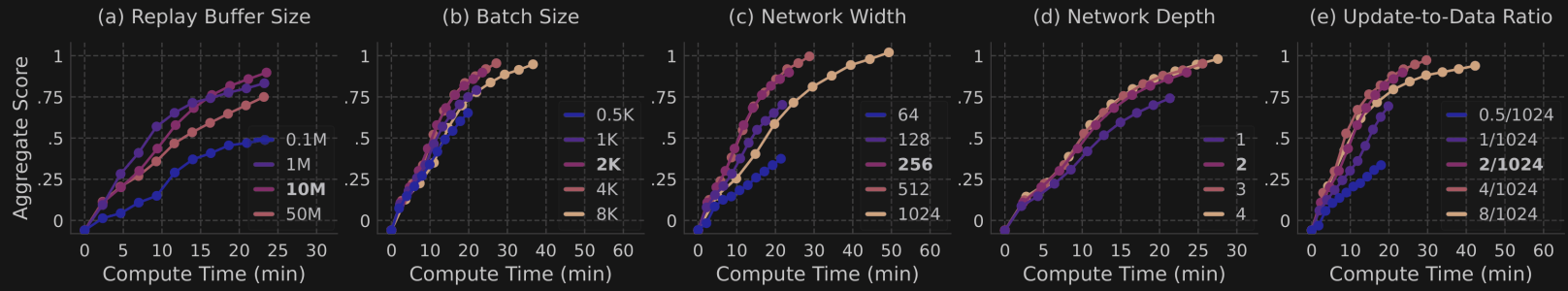


G1 Stair

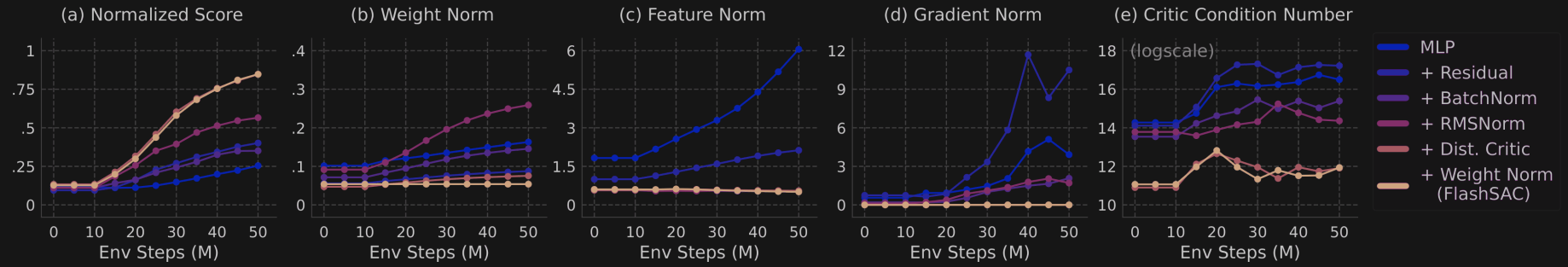


On-Policy vs Off-Policy

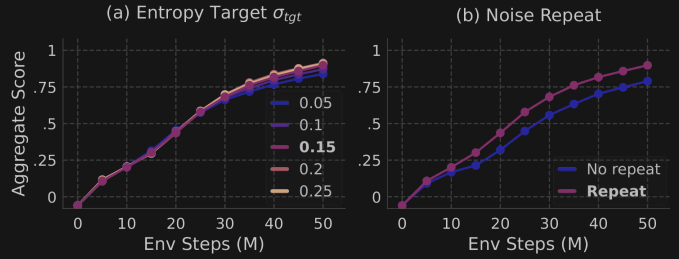
Speed



Stability



Exploration



Conclusion

Alternative to PPO

Higher return, in less time, as the task gets larger.

Scale the model

Keep the data. Update less often, with a bigger batch.

Bound the critic

That is what makes the larger network trainable.